NNOMM FLASH DEALS

High-Concurrency Food Flash Sale Reservation Engine • Staff-Level Master Guide

Target Concurrency: 10,000+ Req/Sec **Latency SLA:** < 35ms P99 **Storage Guarantee:** 0 Oversell Invariant

Core Stack: Kotlin 1.9 • Spring Boot 3.2 • PostgreSQL 15

1 What is Nnomm? The Flash Sale Problem in Plain English

> **🍔 The 50 Wagyu Burger Flash Sale Analogy**
>
> Imagine a top-tier bistro offering **exactly 50 Gourmet Wagyu Burger Vouchers for $5 each** at 12:00:00 PM. At 12:00:00, **20,000 hungry customers** tap "Claim Deal" on their phones within the exact same second.
>
> If your backend software has a concurrency glitch and sells **51 vouchers**, the restaurant faces a physical catastrophe: there are only 50 patties in the kitchen! The 51st guest arrives, gets rejected, demands a refund, leaves furious reviews, and reports fraud.

Nnomm is the backend reservation engine engineered specifically to solve this high-concurrency challenge. It delivers three ironclad guarantees:

1. **Zero Overselling Ever:** Even under 100,000 simultaneous claims, exactly 50 vouchers are issued. Never 51, never 52.
2. **Zero Replay / Double Charges:** If a customer's phone reconnects and fires 10 duplicate requests, they only get 1 reservation and are charged once.
3. **Zero Stock Blackholes:** If a customer claims a meal but closes their browser without paying, their reservation automatically expires after 10 minutes and the meal returns to the available inventory pool.

2 The Tech Stack: WHAT, WHY, and HOW for Every Tool

2.1 Kotlin 1.9 (JVM)

WHAT IS IT?

Modern, statically typed language developed by JetBrains running seamlessly on the Java Virtual Machine (JVM).

WHY CHOSEN?

Compile-Time Null Safety: Eliminates `NullPointerException`. Immutability by default (`val`) prevents race conditions in app memory.

HOW WE USE IT?

We write all controllers, repositories, DTOs, and asynchronous coroutine background workers in clean, concise Kotlin.

2.2 Spring Boot 3.2.3 (Web MVC)

WHAT IS IT?

Enterprise Java/Kotlin web framework running on an embedded Tomcat HTTP server.

WHY CHOSEN?

Predictable Thread-Per-Request: For database-bound systems, WebFlux (reactive) introduces massive debugging complexity. Tomcat MVC offers rock-solid connection stability.

HOW WE USE IT?

Handles HTTP routing, request validation, JWT security filters, and delegates work to database repositories.

2.3 Spring JDBC & HikariCP (Why We Banned Hibernate/JPA!)

WHAT IS IT?

Direct SQL execution via `NamedParameterJdbcTemplate` paired with HikariCP, the fastest connection pool on the JVM.

WHY BANNED JPA?

JPA destroys flash deals: Optimistic locking (`@Version`) causes massive rollback storms (200 threads fighting for 1 unit = 199 exceptions!). JDBC gives 100% control over SQL query plans.

HOW WE USE IT?

We write raw, parameterized SQL queries with zero ORM overhead. Connections are borrowed for just ~1ms to execute atomic updates and returned instantly.

2.4 PostgreSQL 15 (Why Postgres over Mongo or Redis?)

WHAT IS IT?

The world's leading open-source ACID relational database with Multi-Version Concurrency Control (MVCC).

WHY NOT REDIS/MONGO?

Redis is memory-volatile: Crashes before disk sync lose money. **Mongo lacks strict serializable checks:** Risks phantom reads under load. Postgres provides bulletproof ACID on NVMe disk.

HOW WE USE IT?

Enforces check constraints (`CHECK (available_units >= 0)`), partial unique indexes, and executes atomic conditional `UPDATE` statements.

3 Deep Dive: Concurrency & Race Condition Mechanics

> 🏃 **What is Concurrency?**
> Concurrency means **multiple events happening at the exact same moment**. In a computer, when 10,000 requests hit your server, the operating system spreads them across multiple CPU threads. All these threads run in parallel, fighting to read and write the exact same counter in memory or in the database.

3.1 The Time-of-Check to Time-of-Use (TOCTOU) Bug

Here is the naive code that causes real-world businesses to oversell and collapse:

```
// ❌ THE DANGEROUS TOCTOU BUG:
val currentStock = query("SELECT available_units FROM deals WHERE id = ?", dealId) // STEP 1: CHECK
if (currentStock > 0) {
    // 💥 DISASTER WINDOW: 500 other threads also read currentStock = 1 before anyone updates!
    update("UPDATE deals SET available_units = ? WHERE id = ?", currentStock - 1, dealId) // STEP 2: USE
    createClaim(dealId, customerId)
}
```

> 💥 **Why It Fails Under Traffic Surges:**
> Suppose there is only **1 meal left**. Thread A and Thread B run at the exact same millisecond:
>
> - Thread A checks stock: `1 > 0` $ o$ **True!**
> - Thread B checks stock: `1 > 0` $ o$ **True!**
> - Thread A decrements stock to `0` and promises a meal to Customer A.
> - Thread B decrements stock to `-1` and promises a meal to Customer B!
>
> You just promised 2 meals when you only had 1 in stock!

3.2 How Nnomm Wins: The 1-Line Atomic Conditional SQL Decrement

Instead of doing a separate "check" followed by an "update", Nnomm combines both into a **single atomic statement**:

```
UPDATE deals
SET available_units = available_units - 1,
    updated_at = NOW()
WHERE id = :dealId
  AND available_units > 0
  AND status = 'ACTIVE';
```

> 💡 **How PostgreSQL Handles This Under the Hood:**
> When PostgreSQL executes this query, its storage engine locks that specific deal row for just **a few microseconds**:
>
> 1. Transactions line up in PostgreSQL's native lock queue.
> 2. As each transaction evaluates the row, PostgreSQL re-checks the `WHERE available_units > 0` condition against the *latest committed value*.
> 3. The exact millisecond stock reaches 0, the `WHERE` condition fails for all subsequent transactions.
> 4. PostgreSQL updates **0 rows**. The JDBC driver returns `rowsAffected == 0`.
> 5. Our Kotlin code immediately sees `0` and responds with `409 CONFLICT (SOLD_OUT)`.
>
> **Mathematical Result:** Overselling is impossible. Zero retries, zero exception storms, and lock hold times under 1 millisecond!

3.3 Comparison of 5 Concurrency Architectures

Concurrency Pattern	Safety Rating	How It Works	Failure Mode Under Surge
1. Naive Check-Then-Act	**FATAL**	`SELECT` stock then `UPDATE` in application memory.	Severe overselling due to TOCTOU race conditions.
2. JPA `@Version` (Optimistic)	**POOR**	Version number increments on edit; aborts if version mismatch.	Massive exception storms. 200 threads competing for 1 item causes 199 rolled-back exceptions, melting CPU.
3. `SELECT ... FOR UPDATE`	**FAIR**	Acquires exclusive row-level lock before reading.	Requests queue up in database. If 2,000 requests queue, HikariCP pool exhausts and whole API times out.
4. Redis Redlock	**COMPLEX**	Distributed lock across multiple Redis instances.	Network partitions, GC pauses, and clock drift can cause split-brain locks. Still requires DB persistence anyway.
5. Nnomm Pattern: Atomic Conditional SQL	**OPTIMAL**	Single atomic SQL statement evaluated by PostgreSQL tuple lock.	**Zero oversell, zero retries, no distributed lock overhead, microsecond lock hold time.**

4 Deep Dive: Payment Webhooks & Zero-Trust Security

> **What is a Webhook?**
> When a customer pays on Stripe or Adyen, credit card networks take 3 to 15 seconds to process the payment (checking fraud, 3D Secure SMS codes, bank authorization). The user's mobile app cannot keep an open connection waiting that long.
>
> Instead, when the payment finishes, the payment gateway makes an HTTP POST request to our server (`POST /webhooks/payments`) saying: "*Payment succeeded for Claim #7a4b9c! Here is the transaction receipt.*"

4.1 HMAC-SHA256: The Digital Wax Seal

How do we know a malicious attacker didn't send a fake HTTP request to get free meals? The payment gateway and Nnomm share a secret cryptographic key (`webhook_secret`). Before sending the webhook, Stripe computes a cryptographic hash of the entire request body using **HMAC-SHA256** and sends it in the `X-Webhook-Signature` header. When Nnomm receives the request, Nnomm computes the exact same HMAC hash from the raw bytes and compares them. If a single character in the body was tampered with, the hashes will not match!

4.2 Timing Attacks & Constant-Time Evaluation

> **The Timing Attack Vulnerability Explained Simply**
> Standard string comparison in Java/Kotlin (`str1 == str2` or `str1.equals(str2)`) stops immediately on the **first character that doesn't match**.
> Suppose the real secret signature is `f8a3c9...` :
>
> - If an attacker guesses `a00000...` : Java fails on character 1 and returns `false` in **1 nanosecond**.
> - If an attacker guesses `f00000...` : Java matches character 1, fails on character 2, and returns `false` in **2 nanoseconds**!
>
> By measuring response latency with high-precision network probes, an attacker can figure out how many characters were correct and crack the signature byte-by-byte!

Nnomm's Bulletproof Defense: We use Java's `MessageDigest.isEqual(computedHash, receivedHash)` . This method evaluates **all bytes in strictly constant time**, regardless of where mismatches occur. Attackers get zero timing clues.

4.3 Out-of-Order Webhooks & The "Late Payment" Disaster

> **What Happens if a Customer Pays at Minute 11?**
> Customer A claims a meal (10-minute payment countdown). At minute 10:01, Customer A hasn't paid. Our background sweeper expires Claim A and returns the meal to stock. At minute 10:05, Customer B claims that newly freed meal! At minute 11:00, Stripe delivers a delayed webhook saying Customer A's bank payment succeeded!
>
> **How Nnomm handles this:** Nnomm inspects Claim A. Seeing `status == 'EXPIRED'` , Nnomm **refuses to mark it CONFIRMED** (which would create an oversell!). Instead, Claim A transitions to `EXPIRED_NEEDS_REFUND` , and Nnomm dispatches an automated refund command back to Stripe. **The restaurant never oversells, and Customer A gets their money back automatically!**

4.4 Webhook State Transition Matrix

Current Claim Status	Webhook Event	New Claim Status	System Action & Inventory Consequence
`PENDING`	`payment.success`	`CONFIRMED`	Normal success path. Payment recorded. Inventory remains allocated (no change).
`PENDING`	`payment.failed`	`CANCELLED`	Claim cancelled. Deal inventory atomically incremented (+1 available).
`CONFIRMED`	`payment.success` (Duplicate)	`CONFIRMED`	Idempotent replay. Webhook returns `200 OK` immediately without side effects.
`EXPIRED`	`payment.success` (Late Webhook)	`EXPIRED_NEEDS_REFUND`	**Safety Guard:** Stock was already resold. System does NOT oversell; dispatches automated customer refund workflow.

5 Deep Dive: JWT (JSON Web Tokens) & Stateless Security

> **🔑 What is a JWT?**
> A JSON Web Token is a compact, URL-safe string formatted as three parts separated by dots: `Header.Payload.Signature`. It contains authenticated user information (like customer ID and role) cryptographically signed by the server.

1. HEADER
Algorithm & Token Type:
`{"alg":"HS256","typ":"JWT"}`

2. PAYLOAD (Claims)
User Identity & Roles:
`{"sub":"cust_42","role":"CUSTOMER","exp":1718000000}`

3. SIGNATURE
HMAC-SHA256 Hash:
`HMACSHA256(header + "." + payload, secret)`

5.1 Why Traditional Sessions Collapse Flash Deals (And Why JWT Wins)

Authentication Style	How It Works	Behavior Under 10,000 Req/Sec Flash Sale
Stateful Session Cookies	Server stores session ID in database or Redis. On every single API request, the server queries the database: `SELECT * FROM sessions WHERE id = ?`.	**CATASTROPHIC:** 10,000 requests/sec means **10,000 extra database queries/sec** just to check if users are logged in! Connection pools saturate and the database crashes before any claims even execute.
Stateless JWT (Nnomm)	All identity information (customer ID, role, expiration) is packaged inside the token itself, signed cryptographically with a secret key.	**OPTIMAL:** The server verifies the cryptographic signature in CPU memory in **0.05 microseconds with ZERO database calls**! Database connections are preserved 100% for actual inventory transactions.

5.2 Tamper Immunity: Can a Hacker Modify Their JWT?

Suppose a malicious user with `sub: "customer_100"` modifies the payload in base64 to say `sub: "customer_999"` or changes `role: "CUSTOMER"` to `role: "ADMIN"`: When Nnomm's `JwtSecurityFilter` receives the token, it recalculates the HMAC signature using the server's private secret key. Because the payload changed, the calculated signature will not match the signature on the token. The server immediately rejects the request with **401 Unauthorized** before any business logic runs!

5.3 Role-Based Security & Obscurity Protection

- **Customer Access:** Customers can claim deals and inspect their own reservations.
- **Security Through Obscurity:** If Customer Bob tries to view Customer Alice's claim (`GET /claims/{aliceId}`), Nnomm returns **404 Not Found** instead of **403 Forbidden**. Returning 403 would confirm to Bob that Alice's claim ID exists; returning 404 reveals zero information, preventing ID enumeration probing.
- **Admin Access:** Only users with `role: "ADMIN"` can create new flash deals or trigger background sweeps.

6 Deep Dive: Database Transactions & ACID Guarantees

> 🏦 **What is a Database Transaction?**
>
> A transaction is an **"all-or-nothing" execution envelope**. Imagine withdrawing $50 from an ATM. Two things must happen: (1) Your bank account balance decreases by $50, and (2) The ATM machine dispenses the cash. If the cash dispenser jams, you would be furious if your account balance stayed decreased! A database transaction ensures that either *both* actions succeed, or *everything* rolls back to how it was before.

6.1 The 4 ACID Principles Explained in Plain English

ACID Property	What It Means	How Nnomm Enforces It Under High Concurrency
Atomicity (All or Nothing)	A transaction cannot partially execute. If any statement fails, the entire transaction rolls back.	When claiming a deal, decrements to `deals.available_units` and inserts into `claims` happen in one transaction. If the claim insert fails, the stock decrement is rolled back immediately.
Consistency (Rules Never Broken)	The database enforces all constraints, types, and invariants at all times.	The database check constraint `CHECK (available_units >= 0)` and partial unique index `unique_active_claim_per_user_deal` physically prevent illegal states from ever being written.
Isolation (Invisible In-Flight Work)	Transactions running in parallel do not see each other's half-finished, uncommitted data (dirty reads).	Nnomm runs under PostgreSQL's **Read Committed** isolation. Uncommitted decrements or inserts are completely invisible to other threads until committed.
Durability (Survives Power Loss)	Once a transaction commits, the changes are permanent on physical disk, even if the server explodes a millisecond later.	PostgreSQL writes every committed transaction to its **Write-Ahead Log (WAL)** on non-volatile NVMe storage before sending the success confirmation.

6.2 The "Connection Hold-Time" Trap: How Bad Code Kills Databases

The most common mistake in Spring Boot applications is putting `@Transactional` on a service method that makes external network calls:

```
// ❌ THE FATAL TRANSACTION HOLD-TIME MISTAKE:
@Transactional
fun processCheckout(dealId: String, customerId: String) {
    // 1. Thread borrows a database connection from HikariCP pool
    val deal = dealRepository.get(dealId)

    // 💥 DISASTER: Thread holds database connection while waiting 5 seconds for Stripe!
    val paymentResult = stripeClient.chargeCard(deal.price) // 5000ms network call

    // 2. Finally updates database
    claimRepository.confirm(dealId, customerId)
    // 3. Releases database connection
}
```

> 💥 **Why This Crashes Systems:**
>
> If HikariCP has 30 connections in its pool, and 30 requests make external Stripe calls that take 3 seconds each, **all 30 database connections are held hostage waiting for HTTP packets!** Every other user on the entire website gets `ConnectionTimeoutException: Connection is not available`.

> 🛡️ **Nnomm's Architecture: Microsecond Connection Hold Times**
>
> In Nnomm, **database connections are NEVER held across external network calls**:
>
> 1. JWT validation happens entirely in application memory before touching the database.
> 2. Payload hashing and rate limiting happen in application memory.
> 3. A database connection is borrowed **only for the 1-2 milliseconds needed to execute the atomic SQL update**.
> 4. The connection is returned to HikariCP immediately.
>
> Because each connection is held for only ~1ms, a small pool of just 30 connections comfortably serves **over 10,000 requests per second** with average queue wait times below 1 millisecond!

7 The Autonomous Expiry Sweeper (The Stock Recycler)

If 50 users claim all 50 vouchers at 12:00 PM, but 10 of them close their browsers and never pay, those 10 vouchers must not sit locked forever. An autonomous Kotlin Coroutine sweeper runs every 5 seconds to recycle abandoned inventory.

7.1 Multi-Instance Safe Row-Skipping with `SKIP LOCKED`

```
// Sweeper Query: Runs every 5 seconds via Kotlin Coroutines
SELECT id, deal_id
FROM claims
WHERE status = 'PENDING'
  AND expires_at < NOW()
ORDER BY expires_at ASC
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

> 🍽️ **The Restaurant Waiters Analogy**
> `SKIP LOCKED` is like 5 waiters cleaning tables in a dining room. When Waiter 1 starts wiping Table 5, Table 5 is "locked". When Waiter 2 walks by, instead of standing there waiting for Waiter 1 to finish, Waiter 2 **skips Table 5** and cleans Table 6 immediately! Multiple server pods run the sweeper concurrently without blocking each other or causing database deadlocks.

7.2 The 2-Step Atomic Stock Recovery

1. The sweeper marks the stale claim as `EXPIRED`.
2. It atomically increments the deal inventory: `UPDATE deals SET available_units = available_units + 1 WHERE id = :dealId`.
3. The voucher is instantly back on the live menu for other customers to purchase!

8 Staff-Level System Design & Interview Defense (10 Tough Q&As)

Q1 Why did you use PostgreSQL for inventory instead of Redis?

Answer: While Redis is fast, in-memory architectures introduce severe split-brain and durability risks during network partitions or crashes before AOF sync. PostgreSQL 15 on NVMe SSDs executes 10,000+ atomic conditional updates per second with true ACID durability. By doing atomic updates in SQL, we get memory-like speed without risking lost financial state.

Q2 How would you scale this to 100,000 claims/sec for a nationwide Super Bowl flash deal?

Answer: A single PostgreSQL primary maxes out at ~15,000 writes/sec on a single row due to write serialization. To reach 100k req/sec, we implement **Virtual Inventory Sharding**: split the deal into 10 virtual sub-counters (e.g. `deal_1_chunk_0` to `deal_1_chunk_9`). Requests hash their `customer_id % 10` to hit independent row locks, multiplying database throughput by 10x. Additionally, we front reservations with Kafka to smooth incoming traffic into sustained batch updates.

Q3 How does your HikariCP database connection pool prevent starvation under traffic surges?

Answer: Connection starvation happens when threads hold database connections while waiting for external network calls. In Nnomm, connections are never held across network boundaries. JWT validation and payload hashing happen in application memory. The SQL transaction executes a single atomic statement (~1ms) and returns the connection. Thus, a pool of 30 connections comfortably serves thousands of requests per second.

Q4 Why does `GET /claims/{id}` return 404 instead of 403 when an unauthorized user probes it?

Answer: Security through obscurity prevents ID enumeration attacks. If user Bob requests Alice's claim ID and gets `403 Forbidden`, Bob now knows that Claim ID is real and belongs to someone else. By returning `404 Not Found`, we reveal zero information about whether the resource exists.

Q5 How do you perform zero-downtime database schema migrations for tables with millions of rows?

Answer: We enforce the **Expand/Contract Pattern** via Flyway: (1) Expand: Add new columns or indexes non-blockingly (e.g. `CREATE INDEX CONCURRENTLY`). (2) Dual-write: App writes to both old and new columns. (3) Backfill: Async worker migrates historical data. (4) Contract: App reads/writes exclusively to new column; drop old column in next release.

Q6 Why did you choose bare-metal JDBC instead of Spring Data JPA/Hibernate?

Answer: JPA's optimistic locking (`@Version`) causes massive rollback storms under row contention (200 threads competing for 1 unit results in 199 rolled-back exceptions). Furthermore, Hibernate dirty checking and entity snapshots add memory and CPU bloat. Bare-metal JDBC gives 100% control over the microsecond execution plan and guarantees exact row-level locking semantics.

Q7 How does the idempotency engine handle server crashes mid-request?

Answer: We use a 2-phase lease in the `idempotency_keys` table with a 60-second crash lease (`locked_until = NOW() + 60s`). If a node dies mid-transaction, requests within 60s receive `409 REQUEST_IN_PROGRESS` . Once the 60s lease expires, a retry request safely renews the lock and re-executes. If the request previously committed, any retry instantly receives the cached 201 response.

Q8 Why is `MessageDigest.isEqual` used in webhook verification instead of `==` ?

Answer: Standard string equality operators terminate early on the first mismatched byte, creating a timing side-channel attack where hackers measure response times to guess signatures byte by byte. `MessageDigest.isEqual` runs in strictly constant time across all bytes, neutralizing timing attacks.

Q9 How do you ensure multi-region consistency for flash sales?

Answer: Multi-region active-active database writes introduce cross-continent replication latency (~80ms), which destroys flash deal performance. The correct industry approach is **Region-Pinned Inventory**: assign specific deal inventory to specific regional data centers (e.g. 50 units in US-East, 50 units in EU-West). Each region operates autonomously with 1ms latency and zero cross-region locks.

Q10 How does PostgreSQL maintain vacuum performance on hot tables with thousands of updates?

Answer: High-frequency updates on PostgreSQL can cause table bloat if MVCC dead tuples accumulate. We configure PostgreSQL's **Heap-Only Tuples (HOT)** optimization by setting table `FILLFACTOR = 70` on the `deals` table. This leaves 30% free space in each page, allowing updates to modify rows without updating table index pointers, resulting in zero index bloat and minimal autovacuum pressure.